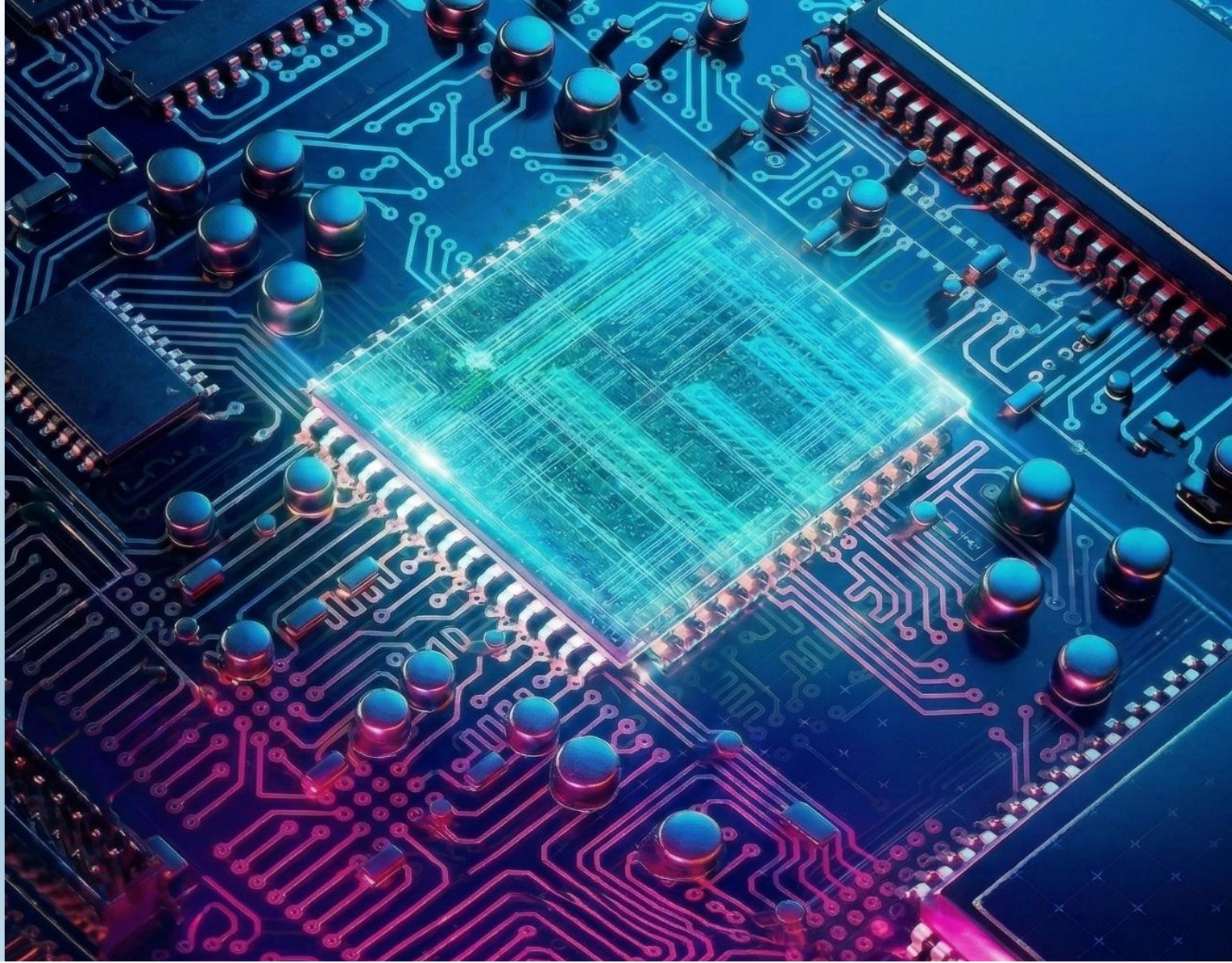




Digital Verification Assignment 4

Name: Mohamed Torki

Alexandria University, Faculty of Engineering
Electronics and Communications Department

Question 1:

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_0	Input A must cover zero, max positive, max negative, and walking-ones patterns when reduction op on A is active.	Randomization under Assignment_2 constraints A toward MAXPOS, MAXNEG, ZERO and walking-ones patterns when red_op_A=1.	bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_1	Input B must cover zero, max positive, max negative, and walking-ones patterns when reduction op on B is active and A reduction is inactive.	Randomization under Assignment_2 constraints B toward walking-ones patterns when red_op_B=1 and red_op_A=0.	bins for ZERO, MAXPOS, MAXNEG, default, and walking-ones (3'b001, 3'b010, 3'b100) iff(red_op_B && !red_op_A).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_2	Opcode must cover all valid operations, the transitions in order, and never take invalid values.	Randomization under Assignment_2 with weighted dist across all opcodes including INVALID_6/7 at lower weight.	bins for shift (SHIFT,ROTATE), arith (ADD,MULT), bitwise (OR,XOR), illegal_bins for (INVALID_6,INVALID_7), transition bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_3	All inputs are randomized with weighted distributions: opcodes equally weighted, bypass and rst favoring deasserted, A/B biased toward boundary values for arithmetic, walking-ones for bitwise, and direction biased for shift/rotate.	Assignment_2 constraint drives dist on opcode, bypass_A/B, rst, A, B, red_op_A/B, direction conditioned on opcode group.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_4	The ALSU must produce correct outputs for all 6 valid opcodes applied in a unique, non-repeating sequence.	Assignment_3 constraint forces my_array to hold all 6 opcodes (OR to ROTATE) uniquely using unique(my_array) and foreach range constraint.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_5	On reset assertion, outputs (out and leds) must be low.	Directed stimulus: rst forced to 1 then 0 twice using assert_reset task before any randomization begins.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_6	All ALSU operations produce correct outputs under fully randomized constrained stimulus across 320 iterations.	320-iteration loop with Assignment_2 constraints active and Assignment_3 constraints disabled. All inputs randomized and driven to DUT each iteration.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_7	All 6 valid opcodes exercised in a unique sequential order within each outer iteration, with bypass and reduction signals deasserted.	Assignment_3 constraints enabled after disabling all constraints. randomize() called once to fill my_array with 6 unique opcodes. rst, bypass, red_op signals forced to 0.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_8	All ALSU operations produce correct outputs under an extended 1000-iteration constrained random stimulus for deeper corner-case coverage.	1000-iteration loop with all constraints disabled except Assignment_2 constraints.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_9	The opcode transition coverage bin (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE) must be explicitly hit in the exact required order.	Obj.opcode set to OR then goes through XOR to ROTATE.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_10	When the ALU is addition or multiplication, A and B should have taken all permutations of maxpos, maxneg and zero	Constrained random	Cross bins: arithmetic op x {A_max, A_min, A_zero} x {B_max, B_min, B_zero} — 9 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_11	When the ALU is addition, c_in should have taken 0 or 1	Constrained random	Cross bins: ADD x {cin_0, cin_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_12	When the ALSU is shifting or rotating, then direction must take 0 or 1	Constrained random	Cross bins: shift/rotate op x {direction_0, direction_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_13	When the ALSU is shifting, then shift_in must take 0 or 1	Constrained random	Cross bins: SHIFT x {serial_in_0, serial_in_1} — 2 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_14	When the ALSU is OR or XOR and red_op_A is asserted, then A took all walking one patterns (001, 010, and 100) while B is taking the value 0	Constrained random	Cross bins: bitwise op x red_op_A=1 x {A_walkingones[0,1,2]} x B_zero — 3 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_15	When the ALSU is OR or XOR and red_op_B is asserted, then B took all walking one patterns (001, 010, and 100) while A is taking the value 0	Constrained random	Cross bins: bitwise op x red_op_B=1 x {B_walkingones[0,1,2]} x A_zero — 3 bins	A checker in the testbench to make sure the output is correct in comparison with a golden model.
ALSU_16	Covering the invalid case: reduction operation is activated while the opcode is not OR or XOR	Constrained random	Illegal bins: red_op_A=1 or red_op_B=1 with arithmetic or shift ops	A checker in the testbench to make sure the output is correct in comparison with a golden model.



Package:

Added these coverpoints

```

cin_cp: coverpoint cin {
    bins cin_0 = {0};
    bins cin_1 = {1};
}

direction_cp: coverpoint direction {
    bins direction_0 = {0};
    bins direction_1 = {1};
}

serial_in_cp: coverpoint serial_in {
    bins serial_in_0 = {0};
    bins serial_in_1 = {1};
}

red_op_A_cp: coverpoint red_op_A {
    bins red_op_A_0 = {0};
    bins red_op_A_1 = {1};
}

red_op_B_cp: coverpoint red_op_B {
    bins red_op_B_0 = {0};
    bins red_op_B_1 = {1};
}

```

```

Cross_cov: cross A_cp, B_cp, ALU_cp, red_op_A_cp, red_op_B_cp,
direction_cp, serial_in_cp, cin_cp {
    option.cross_auto_bin_max = 0;
    //ALSU_10
    bins ARITH_AMP_BMP = binsof(ALU_cp.Bins_arith) &&
binsof(A_cp.A_data_max) && binsof(B_cp.B_data_max);
    bins ARITH_AMP_BMN = binsof(ALU_cp.Bins_arith) &&
binsof(A_cp.A_data_max) && binsof(B_cp.B_data_min);
    bins ARITH_AMP_BZE = binsof(ALU_cp.Bins_arith) &&
binsof(A_cp.A_data_max) && binsof(B_cp.B_data_0);

    bins ARITH_AMN_BMP = binsof(ALU_cp.Bins_arith) &&
binsof(A_cp.A_data_min) && binsof(B_cp.B_data_max);
    bins ARITH_AMN_BMN = binsof(ALU_cp.Bins_arith) &&
binsof(A_cp.A_data_min) && binsof(B_cp.B_data_min);
}

```




```

        bins ARITH_AMN_BZE = binsof(ALU_cp.Bins_arith) &&
        binsof(A_cp.A_data_min) && binsof(B_cp.B_data_0);

        bins ARITH_AZE_BMP = binsof(ALU_cp.Bins_arith) &&
        binsof(A_cp.A_data_0) && binsof(B_cp.B_data_max);
        bins ARITH_AZE_BMN = binsof(ALU_cp.Bins_arith) &&
        binsof(A_cp.A_data_0) && binsof(B_cp.B_data_min);
        bins ARITH_AZE_BZE = binsof(ALU_cp.Bins_arith) &&
        binsof(A_cp.A_data_0) && binsof(B_cp.B_data_0);

        //ALSU_11
        bins ADD_CARRY0      = binsof(ALU_cp) intersect {ADD} &&
        binsof(cin_cp.cin_0);
        bins ADD_CARRY1      = binsof(ALU_cp) intersect {ADD} &&
        binsof(cin_cp.cin_1);

        //ALSU_12
        bins SHFT_ROR_DIR0 = binsof(ALU_cp.Bins_shift) &&
        binsof(direction_cp.direction_0);
        bins SHFT_ROR_DIR1 = binsof(ALU_cp.Bins_shift) &&
        binsof(direction_cp.direction_1);

        //ALSU_13
        bins SHIFT_IN0      = binsof(ALU_cp) intersect {SHIFT} &&
        binsof(serial_in_cp.serial_in_0);
        bins SHIFT_IN1      = binsof(ALU_cp) intersect {SHIFT} &&
        binsof(serial_in_cp.serial_in_1);

        //ALSU_14
        bins BITWISE_PAT_0 = binsof(ALU_cp.Bins_bitwise) &&
        binsof(red_op_A_cp.red_op_A_1) &&
        binsof(A_cp.A_data_walkingones[0]) &&
        binsof(B_cp.B_data_0);
        bins BITWISE_PAT_1 = binsof(ALU_cp.Bins_bitwise) &&
        binsof(red_op_A_cp.red_op_A_1) &&
        binsof(A_cp.A_data_walkingones[1]) &&
        binsof(B_cp.B_data_0);
        bins BITWISE_PAT_2 = binsof(ALU_cp.Bins_bitwise) &&
        binsof(red_op_A_cp.red_op_A_1) &&
        binsof(A_cp.A_data_walkingones[2]) &&
        binsof(B_cp.B_data_0);

        //ALSU_15

```



```

        bins BITWISE_PAT_3 = binsof(ALU_cp.Bins_bitwise) &&
binsof(red_op_B_cp.red_op_B_1) &&
                                binsof(B_cp.B_data_walkingones[0]) &&
binsof(A_cp.A_data_0);
        bins BITWISE_PAT_4 = binsof(ALU_cp.Bins_bitwise) &&
binsof(red_op_B_cp.red_op_B_1) &&
                                binsof(B_cp.B_data_walkingones[1]) &&
binsof(A_cp.A_data_0);
        bins BITWISE_PAT_5 = binsof(ALU_cp.Bins_bitwise) &&
binsof(red_op_B_cp.red_op_B_1) &&
                                binsof(B_cp.B_data_walkingones[2]) &&
binsof(A_cp.A_data_0);

        //ALSU_16
        illegal_bins INVALID_RED_OP_A = binsof(red_op_A_cp.red_op_A_1) &&
(binsof(ALU_cp.Bins_arith) ||
binsof(ALU_cp.Bins_shift));
        illegal_bins INVALID_RED_OP_B = binsof(red_op_B_cp.red_op_B_1) &&
(binsof(ALU_cp.Bins_arith) ||
binsof(ALU_cp.Bins_shift));
    }

```

Testbench, Code Coverage Report, Waveform, Design & Do File:

Same as previous assignment



Functional Coverage Report:

INST \ALSU_pkg::ALSU_class::g1	100.00%	100	100.00...	✓
CVP A_cp	100.00%	100	100.00...	✓
CVP B_cp	100.00%	100	100.00...	✓
CVP ALU_cp	100.00%	100	100.00...	✓
CVP cin_cp	100.00%	100	100.00...	✓
CVP direction_cp	100.00%	100	100.00...	✓
CVP serial_in_cp	100.00%	100	100.00...	✓
CVP red_op_A_cp	100.00%	100	100.00...	✓
CVP red_op_B_cp	100.00%	100	100.00...	✓
CROSS Cross_cov	100.00%	100	100.00...	✓
bin ARITH_AMP_BMP		5	1 100.00...	✓
bin ARITH_AMP_BMN		1	1 100.00...	✓
bin ARITH_AMP_BZE		3	1 100.00...	✓
bin ARITH_AMN_BMP		5	1 100.00...	✓
bin ARITH_AMN_BMN		5	1 100.00...	✓
bin ARITH_AMN_BZE		16	1 100.00...	✓
bin ARITH_AZE_BMP		6	1 100.00...	✓
bin ARITH_AZE_BMN		2	1 100.00...	✓
bin ARITH_AZE_BZE		4	1 100.00...	✓
bin ADD_CARRY0		12	1 100.00...	✓
bin ADD_CARRY1		10	1 100.00...	✓
bin SHFT_ROR_DIR0		10	1 100.00...	✓
bin SHFT_ROR_DIR1		14	1 100.00...	✓
bin SHIFT_IN0		3	1 100.00...	✓
bin SHIFT_IN1		1	1 100.00...	✓
bin BITWISE_PAT_0		89	1 100.00...	✓
bin BITWISE_PAT_1		89	1 100.00...	✓
bin BITWISE_PAT_2		89	1 100.00...	✓
bin BITWISE_PAT_3		55	1 100.00...	✓
bin BITWISE_PAT_4		55	1 100.00...	✓
bin BITWISE_PAT_5		55	1 100.00...	✓
illegal_bin INVALID_RED_OP_...	240	-	-	✓
illegal_bin INVALID_RED_OP_...	261	-	-	✓

Cross Cross_cov	100.00%	100	-	Covered
covered/total bins:	21	21	-	
missing/total bins:	0	21	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin ARITH_AMP_BMP	5	1	-	Covered
bin ARITH_AMP_BMN	1	1	-	Covered
bin ARITH_AMP_BZE	3	1	-	Covered
bin ARITH_AMN_BMP	5	1	-	Covered
bin ARITH_AMN_BMN	5	1	-	Covered
bin ARITH_AMN_BZE	16	1	-	Covered
bin ARITH_AZE_BMP	6	1	-	Covered
bin ARITH_AZE_BMN	2	1	-	Covered
bin ARITH_AZE_BZE	4	1	-	Covered
bin ADD_CARRY0	12	1	-	Covered
bin ADD_CARRY1	10	1	-	Covered
bin SHFT_ROR_DIR0	10	1	-	Covered
bin SHFT_ROR_DIR1	14	1	-	Covered
bin SHIFT_IN0	3	1	-	Covered
bin SHIFT_IN1	1	1	-	Covered
bin BITWISE_PAT_0	89	1	-	Covered
bin BITWISE_PAT_1	89	1	-	Covered
bin BITWISE_PAT_2	89	1	-	Covered
bin BITWISE_PAT_3	55	1	-	Covered
bin BITWISE_PAT_4	55	1	-	Covered
bin BITWISE_PAT_5	55	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin INVALID_RED_OP_B	240	-	-	Occurred
illegal_bin INVALID_RED_OP_A	261	-	-	Occurred

```
==== Instance: /ALSU_pkg
=== Design Unit: work.ALSU_pkg
=====
```

```
Covergroup Coverage:
  Covergroups          1      na      na    100.00%
  Coverpoints/Crosses  9      na      na
  Covergroup Bins      50     50      0    100.00%
```

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

Total Coverage By Instance (filtered view): 100.00%



Question 2:

Q1

```
module Q2_1;
    bit clk;
    logic a, b;
    //method 1
    sequence seq1
        a;
    endsequence

    sequence seq2
        ##2 b;
        //##1 b;
    endsequence

    property p1;
        @(posedge clk) seq1 |-> seq2;
        // @(posedge clk) seq1 |=> seq2;
    endproperty

    pr1 : assert property(p1);
    //method 2
    pr1_ : assert property(@(posedge clk) a |-> ##2 b);
    //pr1_ : assert property(@(posedge clk) a |=> ##1 b);

endmodule
```



Q2

```

module Q2_2;
    bit clk;
    logic a, b, c;

    //method 1
    sequence seq1
        a && b;
    endsequence

    sequence seq2
        ##[1:3] c;
        //##[0:2] c;
    endsequence

    property p2;
        @(posedge clk) seq1 |-> seq2;
        // @(posedge clk) seq1 |=> seq2;
    endproperty

    pr2 : assert property(p1);
    //method 2
    pr2_ : assert property(@(posedge clk) a && b |-> ##[1:3] c;
    //pr2_ : assert property(@(posedge clk) a && b |=> ##[0:2] c;

endmodule

```

Q3

```

module Q2_3;
    bit clk;
    logic b;

    sequence s11b
        repeat(2)@(posedge clk) b;
    endsequence

endmodule

```



Q4

```

module Q2_4_1;
    bit clk;
    logic [2:0] X;
    logic [7:0] Y;

    sequence seq
        $onehot(Y); //A Built-in function that returns true if only one bit was
high
    endsequence

pr3: assert property (@(posedge clk) |-> seq);

endmodule

```

```

module Q2_4_2;
    bit clk;
    logic [3:0] D;
    logic [1:0] Y;
    logic valid;

    //method 1
    sequence seq1
        D == 0;
    endsequence

    sequence seq2
        ##1 valid == 0;
        //valid == 0;
    endsequence

    property p4;
        @(posedge clk) seq1 |-> seq2;
        // @(posedge clk) seq1 |=> seq2;
    endproperty

    pr4 : assert property(p4);
    //method 2
    pr4_: assert property(@(posedge clk) (D == 0) |-> ##1 (valid == 0));
    //pr2_: assert property(@(posedge clk) (D == 0) |=> (valid == 0);

endmodule

```



Question 3:

Testbench:

```
import counter_pkg::*;

module counter_tb (counter_if.TEST c_if);

    counter_class Obj;

    initial begin
        c_if.rst_n = 1;
        c_if.load_n = 0;
        c_if.data_load = 0;
        c_if.up_down = 1;
        c_if.ce = 0;

        Obj = new();

        //COUNTER_6
        for(int i = 0; i < 1000; i++) begin
            @(negedge c_if.clk);
            assert(Obj.randomize());
            c_if.rst_n = Obj.rst_n;
            c_if.load_n = Obj.load_n;
            c_if.up_down = Obj.up_down;
            c_if.ce = Obj.ce;
            c_if.data_load = Obj.data_load;

            @(posedge c_if.clk);
            Obj.count_out = c_if.count_out;
            Obj.cover_counter.sample();
        end
        $stop;
    end
endmodule
```



Package:

```

package counter_pkg;
parameter WIDTH = 4;

class counter_class;
    rand bit rst_n, load_n, up_down, ce;
    rand bit [WIDTH-1:0] data_load, count_out;
    parameter max_count = 2**WIDTH - 1;
    // COUNTER_0
    constraint rst_constraint {
        rst_n dist {0 := 5, 1 := 95};
    }
    constraint load_constraint {
        load_n dist {0 := 70, 1 := 30};
    }
    constraint ce_constraint {
        ce dist {1 := 70, 0 := 30};
    }
    covergroup cover_counter;
        // COUNTER_1
        cp1 : coverpoint data_load iff(!load_n && rst_n);
        // COUNTER_2
        cp2 : coverpoint count_out iff(rst_n && ce && up_down){
            bins count_out_bins_up[] = {[0 : max_count]};
        }
        // COUNTER_3
        cp3 : coverpoint count_out iff(rst_n && ce && up_down){
            bins overflow_bin = (max_count ==> 0);
        }
        // COUNTER_4
        cp4 : coverpoint count_out iff(rst_n && ce && !up_down){
            bins count_out_bins_down[] = {[0 : max_count]};
        }
        // COUNTER_5
        cp5 : coverpoint count_out iff(rst_n && ce && !up_down){
            bins underflow_bin = (0 ==> max_count);
        }
    endgroup

    function new();
        cover_counter = new();
    endfunction
endclass
endpackage

```



Design:

```

module counter (counter_if.DUT c_if);

always @(posedge c_if.clk or negedge c_if.rst_n) begin
    if (!c_if.rst_n)
        c_if.count_out <= 0;
    else if (!c_if.load_n)
        c_if.count_out <= c_if.data_load;
    else if (c_if.ce)
        if (c_if.up_down)
            c_if.count_out <= c_if.count_out + 1;
        else
            c_if.count_out <= c_if.count_out - 1;
end

assign c_if.max_count = (c_if.count_out == {c_if.WIDTH{1'b1}})? 1:0;
assign c_if.zero = (c_if.count_out == 0)? 1:0;

endmodule

```

Do File:

```

Assignments > A4 > Q3 > ≡ run.do
1  vlib work
2  vlog *v +cover -covercells
3  vsim -voptargs=+acc work.counter_top -cover
4  add wave *
5  coverage save counter_top.ucdb -onexit
6  run -all

```



Assertions Module:

```

module counter_sva (counter_if.DUT c_if);

    // COUNTER_7
    property p1;
        @(posedge c_if.clk)
        disable iff(!c_if.rst_n)
        ~(c_if.load_n) | => (c_if.count_out == $past(c_if.data_load));
    endproperty

    // COUNTER_8
    property p2;
        @(posedge c_if.clk)
        disable iff(!c_if.rst_n)
        (c_if.load_n && ~c_if.ce) | => $stable(c_if.count_out);
    endproperty

    // COUNTER_9
    property p3;
        @(posedge c_if.clk)
        disable iff(!c_if.rst_n)
        (c_if.load_n && c_if.ce && c_if.up_down) | => ($past(c_if.count_out) ==
c_if.count_out - 1'b1);
    endproperty

    // COUNTER_10
    property p4;
        @(posedge c_if.clk)
        disable iff(!c_if.rst_n)
        (c_if.load_n && c_if.ce && ~c_if.up_down) | => ($past(c_if.count_out) ==
c_if.count_out + 1'b1);
    endproperty

    // COUNTER_11
    always_comb begin
        if(!c_if.rst_n)
            p5: assert final(c_if.count_out == 1'b0);
    end

    // COUNTER_12
    property p6;
        @(posedge c_if.clk)
        (c_if.count_out == c_if.MAX) | -> c_if.max_count;
    endproperty

```




```

// COUNTER_13
property p7;
  @(posedge c_if.clk)
    (c_if.count_out == 1'b0) |-> c_if.zero;
endproperty

p1_a: assert property (p1);
p1_c: cover property (p1);

p2_a: assert property (p2);
p2_c: cover property (p2);

p3_a: assert property (p3);
p3_c: cover property (p3);

p4_a: assert property (p4);
p4_c: cover property (p4);

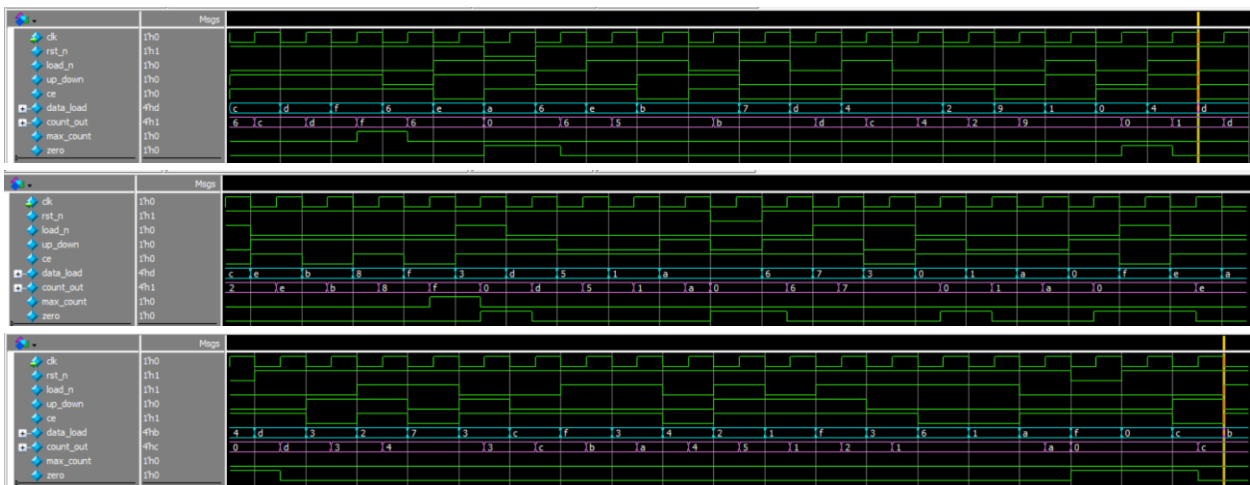
p6_a: assert property (p6);
p6_c: cover property (p6);

p7_a: assert property (p7);
p7_c: cover property (p7);

endmodule

```

Waveform:



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_0	rst_n is deasserted most of the time. load_n is asserted 70% of the time. ce is active 70% of the time.	Randomization under constraints: rst_n dist {0:5, 1:95}, load_n dist {0:70, 1:30}, ce dist {1:70, 0:30}.	-	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_1	When load is asserted and reset is inactive, count_out should take the value of data_load.	Randomization under constraints driving load_n=0 and rst_n=1.	cp1: coverpoint on data_load sampled iff(!load_n && rst_n) — covers all 16 possible load values.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_2	When reset and load are inactive, count_out increments each cycle while ce=1 and up_down=1.	Randomization for up_down and ce, with ce active 70% of the time.	cp2: coverpoint on count_out iff(rst_n && ce && up_down) with individual bins for all values 0 to max_count.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is low	Randomization for up_down, and randomization for enable to be 70% of the time	cp3: transition bin (max_count => 0) on count_out iff(rst_n && ce && up_down).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_4	When reset and load are inactive, count_out decrements each cycle while ce=1 and up_down=0.	Randomization for up_down and ce, with ce active 70% of the time.	cp4: coverpoint on count_out iff(rst_n && ce && !up_down) with individual bins for all values 0 to max_count.	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_5	When counting down, the counter wraps from 0 back to max_count (underflow).	Randomization for up_down=0 and ce=1, running 500 times.	cp5: transition bin (0 => max_count) on count_out iff(rst_n && ce && !up_down).	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_6	All counter behaviours must be exercised under constrained-random stimulus	1000 iterations of constrained random — rst_n 95% high, load_n 30% active, ce 70% active, random data_load	cp1–cp5 sampled via cover_counter.sample() after each posedge	A checker in the testbench to make sure the output is correct in comparison with a golden model.
COUNTER_7	When load is asserted and reset is inactive, count_out should take the value of data_load	Constrained random from COUNTER_6	-	Concurrent assertion P1 checks one cycle after load
COUNTER_8	When load is inactive and ce is low, count_out must not change	Constrained random from COUNTER_6	-	Concurrent assertion P2 checks one cycle after load
COUNTER_9	When load inactive, ce active and up_down high, count_out must increment by 1	Constrained random from COUNTER_6	-	Concurrent assertion P3 checks one cycle after load
COUNTER_10	When load inactive, ce active and up_down low, count_out must decrement by 1	Constrained random from COUNTER_6	-	Concurrent assertion P4 checks one cycle after load
COUNTER_11	When async reset is asserted, count_out must immediately go to 0 in the same delta cycle	Constrained random from COUNTER_6	-	Concurrent assertion P5 checks one cycle after load
COUNTER_12	max_count output must be high whenever count_out equals all-ones	Constrained random from COUNTER_6	-	Concurrent assertion P6 checks one cycle after load
COUNTER_13	zero output must be high whenever count_out equals zero	Constrained random from COUNTER_6	-	Concurrent assertion P7 checks one cycle after load



Assertions:

Assertions									
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Pe
/counter_top/DUT/counter_binded_inst/p5	Immediate	SVA	on	0	1	-	-	-	-
/counter_top/DUT/counter_binded_inst/p1_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/DUT/counter_binded_inst/p2_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/DUT/counter_binded_inst/p3_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/DUT/counter_binded_inst/p4_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/DUT/counter_binded_inst/p6_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/DUT/counter_binded_inst/p7_a	Concurrent	SVA	on	0	1	-	0B	0B	-
/counter_top/TEST/#anonblk#95084642#18#4#/#ublk#9...	Immediate	SVA	on	0	1	-	-	-	-

Cover Directives											
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory
/counter_top/DUT/counter_binded_inst/p1_c	SVA	✓	Off	654	1	Unli...	1	100%		✓	0
/counter_top/DUT/counter_binded_inst/p2_c	SVA	✓	Off	85	1	Unli...	1	100%		✓	0
/counter_top/DUT/counter_binded_inst/p3_c	SVA	✓	Off	89	1	Unli...	1	100%		✓	0
/counter_top/DUT/counter_binded_inst/p4_c	SVA	✓	Off	87	1	Unli...	1	100%		✓	0
/counter_top/DUT/counter_binded_inst/p6_c	SVA	✓	Off	70	1	Unli...	1	100%		✓	0
/counter_top/DUT/counter_binded_inst/p7_c	SVA	✓	Off	146	1	Unli...	1	100%		✓	0

```

=====
=== Instance: /counter_top/DUT/counter_binded_inst
=== Design Unit: work.counter_sva
=====

Assertion Coverage:
  Assertions              7      7      0  100.00%
-----
Name                      File(Line)              Failure Pass
                           Count      Count
-----
/counter_top/DUT/counter_binded_inst/p5
counter_sva.sv(30)              0      1
/counter_top/DUT/counter_binded_inst/p1_a
counter_sva.sv(43)              0      1
/counter_top/DUT/counter_binded_inst/p2_a
counter_sva.sv(46)              0      1
/counter_top/DUT/counter_binded_inst/p3_a
counter_sva.sv(49)              0      1
/counter_top/DUT/counter_binded_inst/p4_a
counter_sva.sv(52)              0      1
/counter_top/DUT/counter_binded_inst/p6_a
counter_sva.sv(55)              0      1
/counter_top/DUT/counter_binded_inst/p7_a
counter_sva.sv(58)              0      1
Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses Coverage
-----
Branches                2      2      0  100.00%

```

```

Directive Coverage:
  Directives              6      6      0  100.00%
-----
DIRECTIVE COVERAGE:
-----
Name                      Design Design  Lang File(Line)  Hits Status
                           Unit  UnitType
-----
/counter_top/DUT/counter_binded_inst/p1_c
counter_sva Verilog  SVA counter_sva.sv(44)
654 Covered
/counter_top/DUT/counter_binded_inst/p2_c
counter_sva Verilog  SVA counter_sva.sv(47)
85 Covered
/counter_top/DUT/counter_binded_inst/p3_c
counter_sva Verilog  SVA counter_sva.sv(50)
89 Covered
/counter_top/DUT/counter_binded_inst/p4_c
counter_sva Verilog  SVA counter_sva.sv(53)
87 Covered
/counter_top/DUT/counter_binded_inst/p6_c
counter_sva Verilog  SVA counter_sva.sv(56)
70 Covered
/counter_top/DUT/counter_binded_inst/p7_c
counter_sva Verilog  SVA counter_sva.sv(59)
146 Covered

```



Functional Coverage Report:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/counter_pkg/counter_class		100.00%				
TYPE cover_counter		100.00%	100	100.00...		✓
CVP cover_counter::cp1		100.00%	100	100.00...		✓
CVP cover_counter::cp2		100.00%	100	100.00...		✓
CVP cover_counter::cp3		100.00%	100	100.00...		✓
CVP cover_counter::cp4		100.00%	100	100.00...		✓
CVP cover_counter::cp5		100.00%	100	100.00...		✓
INST \counter_pkg::counter_class::cover_counter		100.00%	100	100.00...		✓
CVP cp1		100.00%	100	100.00...		✓
CVP cp2		100.00%	100	100.00...		✓
CVP cp3		100.00%	100	100.00...		✓
CVP cp4		100.00%	100	100.00...		✓
CVP cp5		100.00%	100	100.00...		✓

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	5	na	na	na
Covergroup Bins	50	50	0	100.00%

Code Coverage Report:

```
=====  
=== Instance: /counter_top/DUT  
=== Design Unit: work.counter  
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	10	10	0	100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	7	7	0	100.00%

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%



Question 4:

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
CONFIG_0	All registers must initialise to their defined reset values on active-high reset	Directed reset at start of simulation	-	Golden model comparison using reset_assoc array against data_out
CONFIG_1	All registers must store the maximum positive value (0xFFFF) when written	Directed — iterate all 8 addresses with MAXPOS, write enabled	-	Golden model comparison using expected_assoc
CONFIG_2	Registers must hold their value while reading	disable write, iterate all 8 addresses after CONFIG_1 writes	-	Golden model comparison — data_out must match expected_assoc
CONFIG_3	All registers must store the maximum negative value (0x8000) when written	Directed — iterate all 8 addresses with MAXNEG, write enabled	-	Golden model comparison using expected_assoc
CONFIG_4	All registers must store zero (0x0000) when written	Directed — iterate all 8 addresses with ZERO, write enabled	-	Golden model comparison using expected_assoc
CONFIG_5	All registers must correctly store any random 16-bit value when written	Random — Random data written to all 8 addresses sequentially	-	Golden model comparison using expected_assoc
CONFIG_6	Write disabled must not modify any register regardless of data_in value	Directed — write disabled, MAXPOS presented on data_in, all addresses iterated	-	Golden model comparison — data_out must remain equal to expected_assoc

Testbench:

```

module config_reg_tb ();

    bit clk, reset;
    bit write;
    logic [15:0] data_in, data_out;
    logic [2:0] address;

    typedef enum logic [2:0]{adc0_reg, adc1_reg, temp_sensor0_reg,
temp_sensor1_reg,
                                analog_test, digital_test, amp_gain, digital_config}
reg_address;

    reg_address test_reg;

    logic [15:0] reset_assoc [string];
    logic [15:0] expected_assoc [string];

```



```

parameter MAXPOS = 16'hFFFF;
parameter MAXNEG = 16'h8000;
parameter ZERO    = 16'h0000;

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

config_reg DUT(.*);

integer correct_count, error_count;

initial begin
    //config_0
    reset_assoc["adc0_reg"]      = 16'hFFFF;
    reset_assoc["adc1_reg"]      = 16'h0;
    reset_assoc["temp_sensor0_reg"] = 16'h0;
    reset_assoc["temp_sensor1_reg"] = 16'h0;
    reset_assoc["analog_test"]    = 16'hABCD;
    reset_assoc["digital_test"]   = 16'h0;
    reset_assoc["amp_gain"]       = 16'h0;
    reset_assoc["digital_config"] = 16'h1;
    expected_assoc = reset_assoc;

    correct_count = 0;
    error_count   = 0;
    write         = 0;
    address       = 0;
    data_in       = 0;

    //config_1
    assert_reset();

    //config_2
    test_reg = test_reg.first();
    for(int i = 0; i < reset_assoc.size(); i++) begin
        address = test_reg;
        data_in = MAXPOS;
        write   = 1;
        check_result();
        test_reg = test_reg.next();
    end
end

```



```
assert_reset();

//config_3
write    = 0;
test_reg = test_reg.first();
for(int i = 0; i < reset_assoc.size(); i++) begin
    address = test_reg;
    check_result();
    test_reg = test_reg.next();
end

assert_reset();

//config_4
test_reg = test_reg.first();
for(int i = 0; i < reset_assoc.size(); i++) begin
    address = test_reg;
    data_in = MAXNEG;
    write    = 1;
    check_result();
    test_reg = test_reg.next();
end

assert_reset();

//config_5
test_reg = test_reg.first();
for(int i = 0; i < reset_assoc.size(); i++) begin
    address = test_reg;
    data_in = ZERO;
    write    = 1;
    check_result();
    test_reg = test_reg.next();
end

assert_reset();

//config_6
test_reg = test_reg.first();
for(int i = 0; i < reset_assoc.size(); i++) begin
    address = test_reg;
    data_in = $random;
    write    = 1;
    check_result();
    test_reg = test_reg.next();
end
```




```

end

@(posedge clk);
assert_reset();

//config_7
test_reg = test_reg.first();
for(int i = 0; i < reset_assoc.size(); i++) begin
    address = test_reg;
    data_in = MAXPOS;
    write    = 0;
    check_result();
    test_reg = test_reg.next();
end
$display("Simulation done: Correct = %0d | Errors = %0d", correct_count,
error_count);
$stop;
end

task check_result;

    reg_address current_reg;
    current_reg = reg_address'(address);

    if(reset == 1'b1) begin
        test_reg = test_reg.first();
        for(int i = 0; i < reset_assoc.size(); i++) begin
            address = test_reg;
            @(negedge clk);
            if(reset_assoc[test_reg.name()] != data_out) begin
                error_count++;
                $display("%t: Error in Reset: address = 0x%0h | data_out =
0x%0h | data_exp = 0x%0h",
                        $time,address , data_out,
reset_assoc[test_reg.name()]);
            end
            else
                correct_count++;
            test_reg = test_reg.next();
        end
    end
end

else if(write == 1'b1) begin
    @(negedge clk);
    if(data_out != data_in) begin

```



```

        error_count++;
        $display("%t: Error in Write: address = 0x%0h | data_out = 0x%0h
| data_in = 0x%0h",
                $time, address, data_out, data_in);
    end
    else begin
        correct_count++;
        expected_assoc[current_reg.name()] = data_in;
    end
end

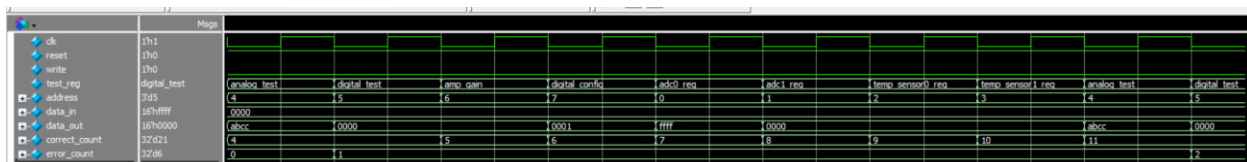
else begin
    @(negedge clk);
    if(data_out != expected_assoc[current_reg.name()]) begin
        error_count++;
        $display("%t: Error in Read : address = 0x%0h | data_out = 0x%0h
| expected = 0x%0h",
                $time, address, data_out,
expected_assoc[current_reg.name()]);
    end
    else
        correct_count++;
    end
endtask

task assert_reset;
    reset = 1;
    check_result();
    expected_assoc = reset_assoc;
    reset = 0;
endtask
endmodule

```

Bugs alongside Waveforms:

Bug in `analog_test` register at Resets to **ABCC** instead of **ABCD**



Register	Value	Register	Value	Register	Value	Register	Value	Register	Value	Register	Value	Register	Value	Register	Value
analog_test	15	digital_test	16	analog_test	17	digital_test	18	analog_test	19	digital_test	20	analog_test	21	digital_test	22
0000		0000		0000		0000		0000		0000		0000		0000	
abcc		0000		0001		ffff		0000		0000		abcc		0000	
0		15		16		17		18		19		20		21	
0		15		16		17		18		19		20		21	



Bug in `temp_sensor0_reg` register is **shifted left by one bit**

	Msgs		
clk	1'h0		
reset	1'h0		
write	1'h1		
test_reg	digital_test		
address	3'd5	temp_sensor0_req	
data_in	16'hffff	2	
data_out	16'h0000	0001	
correct_count	32'd21	0000	0002
error_count	32'd5	21	
		4	

	Msgs		
clk	1'h0		
reset	1'h0		
write	1'h1		
test_reg	digital_config	temp_sensor0_req	
address	3'h7	2	
data_in	16'h5212	d609	
data_out	16'h0001	0000	ac12
correct_count	32'd56	54	
error_count	32'd23	20	

`temp_sensor1_reg` register **doesn't reset**

	Msgs		
clk	1'h1		
reset	1'h1		
write	1'h1		
test_reg	temp_sensor1_reg	temp_sensor1_req	adc0_req
address	3'd3	3	0
data_in	16'hffff	ffff	
data_out	16'hffff	0000	ffff
correct_count	32'd18	14	15
error_count	32'd2	2	

	Msgs		
clk	1'h1		
reset	1'h1		
write	1'h1		
test_reg	temp_sensor1_reg	adc1_req	temp_sensor0_req
address	3'd3	1	2
data_in	16'hffff	ffff	
data_out	16'hffff	0000	ffff
correct_count	32'd18	16	17
error_count	32'd2	18	

Accessing `digital_test` register **always yields to 0**

	Msgs		
clk	1'h1		
reset	1'h0		
write	1'h0		
test_reg	digital_test	digital_test	amp_gain
address	3'd5	5	6
data_in	16'hffff	ffff	
data_out	16'h0000	0000	0001
correct_count	32'd21	18	19
error_count	32'd6	4	

	Msgs		
clk	1'h1		
reset	1'h0		
write	1'h0		
test_reg	digital_test	digital_test	temp_sensor0_req
address	3'd5	5	2
data_in	16'hffff	ffff	
data_out	16'h0000	0000	0002
correct_count	32'd21	20	21
error_count	32'd6	5	6

Bug in the **MSB** of `digital_config` register at **writing**

	Msgs		
clk	1'h0		
reset	1'h0		
write	1'h0		
test_reg	digital_config	digital_config	
address	3'd7	7	
data_in	16'h8000	8000	
data_out	16'h0000	0001	0000
correct_count	32'd22	22	
error_count	32'd7	6	

	Msgs		
clk	1'h0		
reset	1'h0		
write	1'h1		
test_reg	digital_config	digital_test	amp_gain
address	3'h7	5	6
data_in	16'h7fff	ffff	
data_out	16'h7fff	0001	7fff
correct_count	32'd12	11	12
error_count	32'd3	2	3




```

logic [15:0] reset_assoc [string];
logic [15:0] expected_assoc [string];

parameter MAXPOS = 16'hFFFF;
parameter MAXNEG = 16'h8000;
parameter ZERO    = 16'h0000;

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

config_reg DUT(.*);

integer correct_count, error_count;

initial begin
    reset_assoc["adc0_reg"]      = 16'hFFFF;
    reset_assoc["adc1_reg"]      = 16'h0;
    reset_assoc["temp_sensor0_reg"] = 16'h0;
    reset_assoc["temp_sensor1_reg"] = 16'h0;
    reset_assoc["analog_test"]    = 16'hABCD;
    reset_assoc["digital_test"]   = 16'h0;
    reset_assoc["amp_gain"]       = 16'h0;
    reset_assoc["digital_config"] = 16'h1;
    expected_assoc = reset_assoc;

    correct_count = 0;
    error_count   = 0;
    write         = 0;
    address       = 0;
    data_in       = 0;

    assert_reset();

    // BUG 1
    assert_reset();

    // BUG 2
    test_reg = temp_sensor1_reg;
    address  = temp_sensor1_reg;
    data_in  = MAXPOS;
    write    = 1;

```



```
check_result();
assert_reset();

// BUG 3
test_reg = temp_sensor0_reg;
address  = temp_sensor0_reg;
data_in  = 16'h0001;
write    = 1;
check_result();

// BUG 4
test_reg = digital_test;
address  = digital_test;
data_in  = MAXPOS;
write    = 1;
check_result();
write    = 0;
check_result();

// BUG 5
test_reg = digital_config;
address  = digital_config;
data_in  = 16'h8000;
write    = 1;
check_result();
write    = 0;
check_result();

// BUG 6
test_reg = adc0_reg;
address  = adc0_reg;
data_in  = 16'hFFFF;
write    = 1;
check_result();
data_in  = 16'h7FFF;
write    = 1;
check_result();
write    = 0;
check_result();

// BUG 7
test_reg = adc1_reg;
address  = adc1_reg;
data_in  = 16'h8000;
write    = 1;
```



```

    check_result();
    write    = 0;
    check_result();
    data_in  = 16'h0080;
    write    = 1;
    check_result();
    write    = 0;
    check_result();

    // BUG 8
    test_reg = amp_gain;
    address  = amp_gain;
    data_in  = 16'hBEEF;
    write    = 1;
    check_result();
    repeat(8) @(posedge clk);
    write    = 0;
    check_result();
    $display("Simulation done: Correct = %0d | Errors = %0d", correct_count,
error_count);
    $stop;
end

task check_result;

    reg_address current_reg;
    current_reg = reg_address'(address);

    if(reset == 1'b1) begin
        test_reg = test_reg.first();
        for(int i = 0; i < reset_assoc.size(); i++) begin
            address = test_reg;
            @(negedge clk);
            if(reset_assoc[test_reg.name()] != data_out) begin
                error_count++;
                $display("%t: Error in Reset: address = 0x%0h | data_out =
0x%0h | data_exp = 0x%0h",
                        $time,address , data_out,
reset_assoc[test_reg.name()]);
            end
        end
        else
            correct_count++;
            test_reg = test_reg.next();
        end
    end
end

```



```

        else if(write == 1'b1) begin
            @(negedge clk);
            if(data_out != data_in) begin
                error_count++;
                $display("%t: Error in Write: address = 0x%0h | data_out = 0x%0h
| data_in = 0x%0h",
                        $time, address, data_out, data_in);
            end
        else begin
            correct_count++;
            expected_assoc[current_reg.name()] = data_in;
        end
    end

    else begin
        @(negedge clk);
        if(data_out != expected_assoc[current_reg.name()]) begin
            error_count++;
            $display("%t: Error in Read : address = 0x%0h | data_out = 0x%0h
| expected = 0x%0h",
                    $time, address, data_out,
expected_assoc[current_reg.name()]);
        end
    else
        correct_count++;
    end
endtask

task assert_reset;
    reset = 1;
    check_result();
    expected_assoc = reset_assoc;
    reset = 0;
endtask
endmodule

```

